Gunter & Scott 1990 — Inventory of Definitions and Theorems

Source: **C. A. Gunter and D. S. Scott, "Semantic Domains,"** *Handbook of Theoretical Computer Science* Vol. B, North-Holland, 1990, pp. 633–674 (`../papers/Gunter Scott 1990.pdf`).

The work list for the Lean formalization: every definition and every one of the paper's **30 numbered results** (Theorems / Lemmas / Proposition 1–30), in paper order, matched to its Lean equivalent.

Progress (as of r0036, 2026-0807)

#	Quantity	Done	Remaining	Of
1	Definitions to define	**all ≈13** — the computable function landed in r0031; see the note below on `D∞`	0	≈13
2	**Numbered** results complete	**24** (Thm 1, Thm 3, Lem 4, Lem 5, Thm 6, Thm 7, Lem 8, Lem 9, Lem 10, Thm 11, Thm 12, Lem 13, **Thm 14**, Prop 15, Lem 17, Lem 19, Lem 20, Thm 21, Thm 22, Lem 23, Lem 24, Thm 25, Thm 26, **Thm 27**) — the two in bold landed in r0036, and Thm 27 is now *unconditional*; three of the 24 carry a qualification, see row 2c	**4** (Thm 18, Lem 28, Thm 29, Lem 30 — Thm 18's steps 2 and 3 are proved, Lem 28 is 2 of 9 at the paper's own notion, Thm 29's first sentence is proved and `V` is built with `V ≅ V⁺`; see row 2d)	**29**

#	Quantity	Done	Remaining	Of
2a	— **resolved by refutation**	**Thm 16** — fully characterized as of r0034. The algebraic-lattice conjunct is proved (r0028); the `Fp(D) ↪ (D → D)` embedding conjunct is **false**, kernel-checked (r0032); and the conjunct **does** hold under a named hypothesis, proved as `Section62.thm16_positive` with `thm16_positive_isEmbeddingProjectionPair` (r0034). The result is settled in all three directions, but it is still not a proof of the paper's sentence, so it is counted in neither column. The row-2 arithmetic is therefore 24 + 4 + 1 = 29	—	—

#	Quantity	Done	Remaining	Of
2c	**— qualifications on three of the 24**	**Lem 9**: four conjuncts hold as printed; items 3 and 5 are **false as printed** and are kernel-checked negations (`lem9_3_printed_false`, `lem9_5_printed_false`), with the corrected laws proved. **Lem 17**: 10 of 10, but the → and ∘→ conjuncts carry `[BoundedComplete β]` from the step-function decomposition — stronger than the paper states, and §6 exists precisely to avoid bounded completeness. **Thm 26**: proved with `hs : ∀ i, 0 < s i`; the theorem is **false** for a signature admitting arity 0, which the paper explicitly allows — that argument is prose in the docstring, *not* kernel-checked. **Lem 10** carries no qualification	—	—

#	Quantity	Done	Remaining	Of
2d	**— status of the four remaining**	**Thm 18** is the development's only remaining `sorry`. Its five-step route (Jung 1989) has steps 2 and 3 proved in r0036 as `JungSFP.lemma213`, `thm214` and `lemma217`, and step 5 since r0028; steps 1 and 4 remain. **Lem 28** lists **nine** operators — `→, ↛, ×, ⊗, +, ⊕, ()⊥, ()♯, ()♭` — and its "representable" means **p-representable over `Fp(U)`**: 2 of 9 (`×`, `()⊥`) are proved at that notion in r0036, and the three r0034 proofs at the closure reading are kernel-checked *not* to transfer. **Thm 29** splits: the first sentence is proved, and r0036 built `V` with `Domain V`, `IsBifinite V` and `V ≃o V⁺`; the universality extension step remains. **Lem 30** is over `V` and lists **ten** operators — Lemma 28's nine plus `()♮` — and remains 0 of 10	—	—
3	**Unnumbered prose claims** proved	**12**	—	—
4	Mathlib foundations reused	12	—	12

#	Quantity	Done	Remaining	Of
5	Modules / lines / theorems	**66 / 23596 / 1119** — measured by `scripts/counts.sh` with r0036 merged	—	—
6	`sorry` in the development	—	**1**, in 1 file: `thm18` in `Skeleton/Section6.lean`. r0034 took this from 8 to 2 and r0036 from 2 to 1 by proving Theorem 14. Every other statement in the development is kernel-checked, and nothing in the development depends on `thm18` — the only mention outside its own file and `Section62.lean`'s obstruction write-up is a docstring reference, so no completed result routes through `sorryAx`	—

Round r0036. Five agents at the six then-open numbered results; all five landed and merged. `sorry` 2 → 1; numbered results 22 → 24 of 29; 66 modules, 23596 lines, 1119 theorems. Build 1217 jobs, 0 errors, 0 diagnostics, 0 warnings beyond `sorry`. Composition check: all five new modules import into one environment with no name clash, and every headline theorem depends only on `[propext, Classical.choice, Quot.sound]`.

#	Stream	Landed
1	`SFP.lean`	**Theorem 14** in both directions — the last `sorry` but one. Of r0034's four measured gaps, two were real (the bridge `im(p_N) = N` for finite normal `N`, and two finite-directed-set lemmas) and **two were false constraints**: the `Fp(D)` machinery is never entered, because on a finite image `im(p) ∩ K(D) = im(p)`; and leastness of `id` is proved directly in `ScottHom α α`, not transported from `↥(Fp α)`. One ingredient no earlier note had listed: `M` must be **nonempty**, since `IsCompactElement` quantifies over nonempty directed sets while Mathlib's `DirectedOn` is vacuous on `∅`
2	`JungSFP.lean`	**Theorem 18's steps 2 and 3** — `lemma213`, `thm214` (the bifurcation) and `lemma217` (the uncountable family). What unblocked three failed rounds was not a tactic but a missing lemma: the development states minimal upper bounds *relative to* `K(D)`, while every one of Jung's minimality steps applies them to a bound not known to be compact. `isCompactElement_of_minimal_upperBounds` (Jung's Prop 1.9 for `IsAlgebraic`) bridges the two, and without it `f_A`'s monotonicity is false. **`IsLDomain` proved unnecessary** — Lemma 2.17 uses only condition (vii) of Jung's Theorem 2.10, so `HasTwoMubBelow`/`HasAtMostOneMubBelow` are defined directly and the six equivalences the route never passes through are not formalized

#	Stream	Landed
3	`Atomless.lean`	**Theorem 27, unconditionally.** `IsNormallyRepresented ↥(compacts D)` is proved, so `Atomless.thm27` carries no hypothesis. **Neither Vaught's theorem nor a Boolean algebra is needed** — two claims in `Dyadic.lean`'s docstring were false and are corrected. What Theorem 27 consumes is three properties of a map `ψ : K(D) →` U_0: order embedding into the superset order, `ψ ⊥ = [0,1)`, and `ψ a ∩ ψ b` empty when `{a,b}` is unbounded and `ψ(a ⊔ b)` when bounded. The third is also what makes `range ψ` normal, so the paper's unproved final clause is four lines. `Legal` demands that a join drag in no earlier `enum i` the branch does not carry, which is what keeps `ψ a` a finite union of intervals
4	`PRep.lean`	**Lemma 28 at the paper's own notion** — `×` and `()⊥` p-representable over `Fp(U)`, plus `isFinitaryProjection_sSup`, the keystone giving pointwise least upper bounds in `Fp(D)`. The three r0034 conjuncts are kernel-checked **not** to transfer: `Combinator.Retracts` gives `id ⊑ gr ∘ fn` where the projection scheme needs `gr ∘ fn ⊑ id`, and `gr_fn_eq_of_both` proves holding both forces `U ≅ V`. Position 3 of 9 at a wrong notion → **2 of 9 at the right one**. `⊗` and `⊕` are no longer refuted: a projection has `p ⊥ = ⊥`, so r0034's three-chain counterexample does not apply
5	`Colimit.lean`	**`V`** with `Domain V`, `IsBifinite V` and **`V ≃o V`**$^{+}$ — Theorem 29's fixed point, built as the `Antisymmetrization` of a pre-order on `Σ n, Stg n` with `Nat.leRecOn` transport and zero casts. Lemma 30 is statable for the first time

Three corrections to the r0036 plan, all from agents reading the PDF. The plan's Lemma 28 operator list was wrong in two places — it inserted `()♮`, which §7.4 opens by saying *cannot* be representable over `U`, and dropped `↠`, the strict function space; `CombinatorRep`'s r0034 docstring already had the correct nine and the plan drifted from the file. Lemma 30 has **ten** conjuncts, not nine — Lemma 28's nine plus `()♮`, which is the whole point of §7.4. And the plan's route for `V`, taken from `BifiniteUniversal.lean` and from §7.4's own sentence, says the colimit is along `eta`; **that colimit is not a fixed point of `M`**, kernel-checked as `Colimit.stgEmb_ne_mk_eta` — a third printed defect in §7.4, alongside the two r0034 recorded. The correct chain applies `M` to the previous connecting map; the two first differ at stage 1→2, where both values lie among §7.4's five elements of `I`$^{++}$, so Figure 4 does not discriminate them. Stage sizes are unaffected, so the 1, 2, 5, **20** count still selects `MPair.le` over the Smyth reading's 21.

Round r0034. Six agents, all landed and merged. `sorry` 8 → 2; numbered results 18 → 22 of 29; 61 modules, 19497 lines, 906 theorems. Build 1137 jobs, 0 errors, 0 warnings beyond `sorry`. Composition check: all fourteen new modules import into one environment with no name clash, and every headline theorem depends only on `[propext, Classical.choice, Quot.sound]`.

#	Stream	Landed
1	`Isomorphism/` (6 modules)	**Lemma 9** — all six laws as named `≃o`, plus `lem9_3_printed_false` and `lem9_5_printed_false`, kernel-checked negations of the two misprints. `Skeleton/Recovered.lean` 7 `sorry` → 1
2	`ClosureProperties/` (4 modules)	**Lemma 10 at 7 of 7** and **Lemma 17 at 10 of 10**, each now stated as *one* theorem — a conjunction over the paper's own operator list — so the conjunct count is kernel-checked rather than prose. Also `liftIsAlgebraic/liftDomain`: `D⊥` was only a cpo, so `D + E` had not been statable
3	`Dyadic.lean`	§7.3's **universal domain `U`** as the ideal completion of the dyadic half-open intervals under *superset*, with `K(U)`, `Domain U` by `inferInstance` from Theorem 11, and `BoundedComplete U`; **Theorem 27** proved from `IsNormallyRepresented`
4	`Combinator.lean`, `CombinatorRep.lean`	**Theorem 26** — the Engeler-style embedding result, *not* equation-solving; 3 of Lemma 28's operators over an abstract `Retracts` interface; the counterexample refuting the closure reading of `⊗` and `⊕`
5	`BifiniteUniversal.lean`, `PRepresentable.lean`	**Theorem 29's first sentence**; **p-representability** over `Fp(U)` with `eq_id_of_mem_Fp_of_mem_Fc` proving it distinct from `IsRepresentable` over `Fc(U)`; two kernel-checked defects in §7.4's printed pre-ordering
6	`Section62.lean`	**`thm16_positive`** and its embedding–projection pair; the **Theorem 18 obstruction**, written as the deliverable — `thm18` untouched

Four corrections to the r0034 plan, all found by agents reading the PDF rather than the plan. Theorem 26 is an Engeler-style embedding result, not "combinators solving a signature's equations"; Lemma 28 lists nine operators at p-representability over `Fp(U)`, not seven at the closure reading; Theorem 14's obstacle is mathematical (Plotkin's SFP characterization), not definitional as the plan asserted; and `prop15`/`thm22` are not on Theorem 27's route at all — §7.3 cites no earlier result of the paper.

Round r0032. Five agents; three landed, two were still running when this was written.

#	Stream	Landed
1	`Universality.lean`	**Lemma 24** and **Theorem 25**, with `thm25_powerset`: `P N` is universal. Proved at *cpo* strength — no step spends algebraicity or countability, so it is stronger than the paper's statement
2	`IdealCompletion.lean`, `Powerdomain/BoundedComplete.lean`	the **`idealSup` repair**, and bounded completeness reinstated for `D♭` and `D♯`
3	`FinitaryProjectionEmbedding.lean`	**Theorem 16's embedding conjunct refuted**
4	`ContinuousAlgebra.lean`	**Theorem 12** for all three powerdomains, existence *and* uniqueness, 1254 lines, 0 `sorry`
5	`Skeleton/Recovered.lean`, `docs/StatementRecovery.md`	**Lemma 9 and Theorem 14 recovered** from the PDF — both were recorded as "not statable"

`pdftotext` drops glyphs, and four inventory rows undercounted because of it. The paper's 18 embedded Type 3 fonts carry `Custom` encoding with no `ToUnicode` map: codes ≥ `0x20` leak through as ASCII, and **codes below `0x20` vanish silently**. Those are standard TeX positions, pinned by the leaked ASCII (`0x21`→→, `0x3F`→⊥, `0x76`→⊑, all `cmsy10`), so decoding the content stream against `cmsy10`/`cmr10`/`cmmi10` recovers the text character for character; a `pdftocairo` rendering confirms it independently. Consequences:

- **+ and ⊕ are different operators, and what this development has is ⊕.** `lem10_sum` and `lem17_sum` range over `CoalescedSum`, so they discharge the ⊕ conjuncts, not the + ones earlier drafts credited them with. + should be cheap: `D + E = D⊥ ⊕ E⊥`.
- **The strict arrow is two glyphs**, `openbullet` + `arrowright`; both → and ∘→ extract as `!`, so every earlier extraction conflated two function spaces.
- **Lemma 17's three powerdomain conjuncts `D♮, D♯, D♭` were never stated**, though all three powerdomains have existed since r0029.
- **Theorem 12's base theory is `T♮`** (natural), not an undecorated `T`: `pdftotext` renders ♮/♯/♭ as `\/]/[`, losing the accidental.
- **Two of Lemma 9's conjuncts are misprints** — items 3 and 5 are *false as printed*, refuted on `D = E = Prop`, `F = Prop × Prop` by cardinality (10 vs 8, and 5 vs 3). The corrections are forced, and item 3's is the universal property of ⊕ that the paper states three pages earlier. Recorded in `docs/StatementRecovery.md`; not yet put under the kernel.

Theorem 16's second conjunct is false. The paper asserts the inclusion `i : Fp(D) ↪ (D → D)` is an embedding. Against a five-element bifinite domain with two incomparable minimal upper bounds — one that satisfies `thm16`'s first conjunct, so the refutation isolates the second — there are two incomparable maximal finitary projections below `f = λx. m₁` and no greatest one, while any monotone section would have to produce one. The sketch's error is exact: `p ⊑ f ⟺ im(p) ∩ K(D) ⊆ S_f`, so what is needed is a normal set **contained in** `S_f`, whereas the paper takes the least normal set **containing** it. Those agree only when `S_f` is already normal. No choice of `N_f` repairs it. A positive statement remains available: the conjunct holds when every `S_f` has a greatest normal subposet, which bounded complete domains satisfy.

The `idealSup` repair, closing the third instance of one defect. `idealSup` now branches on `Order.IsIdeal (genIdeal S)` — the ideal *generated by* the union — rather than on the union itself being an ideal. `genIdeal_eq_sUnion_of_isIdeal` proves the two agree wherever the old guard fired, so `lubOfDirected` and `mem_sSup_iff` kept their statements *and their proofs*, and Theorem 11 is byte-identical. `not_boundedComplete_{hoare,smyth,plotkin}` are retired, and `instBoundedCompleteHoare` (no bounded-completeness hypothesis needed) and `instBoundedCompleteSmyth` take their place. The witness is kept as three live lemmas rather than a docstring: the old guard still fails on it, and the repaired `sSup` returns the true least upper bound. A docstring cannot detect a regression.

Round r0029 ran four agents in parallel and closed the definition list.

#	Stream	Landed
1	`Powerdomain/Hoare.lean`	the Hoare (lower) powerdomain: `Pf` the finite non-empty subsets of `K(D)`, the lower pre-order, and `Domain` from Theorem 11
2	`Powerdomain/Smyth.lean`	the Smyth (upper) powerdomain, same shape, dual orientation
3	`Powerdomain/Plotkin.lean`	the Plotkin (convex) powerdomain under the Egli–Milner pre-order
4	`RecursiveDomain.lean`	the recursive domain equation, two formalizations of *universal domain*, and **Theorem 21** — plus `recursiveDomain_funSpace`, the reflexive domain `D ≅ (D → D)`

There is no `D∞` to build. Earlier drafts of this inventory listed `D∞` (inverse limit) as an outstanding definition. Reading §7 directly refutes that: the section raises the chain $T_0 \to^{e_0} T_1 \to^{e_1} T_2 \to \cdots$, says "This is all very informal, however; how are we to make this idea mathematically precise…?", and answers with §7.1, *Solving domain equations with closures*. `D ≅ D → D` is reached from Theorem 21 and Lemma 23, and the limit is taken inside `Fc(U)`. What stands in `D∞`'s place is `Recursive.Solves` / `IsSolvable`, and the two formalizations of universal domain — `IsUniversal` (every domain of the class is a closure of `U`) and `IsUniversalRetract` (every one is a retract), which the paper states as two different sentences.

`Pf` is the finite *non-empty* subsets. The paper defines `Pf(S)` that way and reserves `P̄ f(S)` for the version including `∅`; all three powerdomains are built over `Pf`. The distinction is load-bearing and in opposite directions for the three orderings: under the Hoare order `∅ ⊑ v` holds vacuously, so admitting `∅` would add a point strictly below `{⊥}`; under the Smyth order `∅` is a **top**, so it would add a spurious maximum; under Egli–Milner `∅` is comparable to nothing but itself, destroying `OrderBot`. The orchestrator's brief said "finite subsets"; each agent read the PDF and corrected it.

Namespace per agent worked. Every r0029 declaration lives in `ScottDomains.{Hoare, Smyth, Plotkin, Recursive}`. Four agents, **zero** name collisions — against two in r0028, when five agents shared one namespace.

Round r0031 ran four agents and closed the definition list.

#	Stream	Landed
1	`ComputableFunction.lean`	the paper's **computable function**, on Mathlib's `REPred` — the last definition
2	`Powerdomain/BoundedComplete.lean`	**Lemma 13** in the paper's own wording, and a refutation: see the `idealSup` defect below
3	`Powerdomain/Universal.lean`	`isRepresentable_prod` — the product operator is representable over `P N`, **the hypothesis Lemma 24 lacked**; plus `D ≅ D × D`
4	Theorem 18	still running at the time of writing

The `idealSup` defect — the same class, a third time. `IdealCompletion.idealSup` branches its `dite` on `Order.IsIdeal (⋃₀ …)`. That is the membership condition for the *union*, but the union is the least upper bound only when the family is **directed**; for a merely **bounded** family the least upper bound is the ideal *generated by* the union. So `sSup` returns `⊥` on a bounded non-directed family — kernel-checked in `P N` with the incomparable compacts `{0}` and `{1}`, bounded above by `{0,1}`. This follows `ScottHom` (r0006–r0011) and `Smash` (r0027), and the r0031 plan asserted the guard was correct, which was wrong. Consequently `not_boundedComplete_hoare`, `_smyth` and `_plotkin` are proved: **no `BoundedComplete` instance can be claimed for any ideal completion until `idealSup` is repaired**, by branching on the *generated* ideal rather than on the union being one.

Lemma 13 is nonetheless true and proved, in the paper's own wording (`BddAbove S → ∃ I, IsLUB S I`, with the least element from `OrderBot`): `D♭` (Hoare) needs only the union, so it is bounded complete for **every** domain and the hypothesis is never consumed; `D♯` (Smyth) spends both hypotheses. There is no `lem13_plotkin` to prove — Lemma 13 names only `D]` and `D[`, and Egli–Milner has no join for a bounded pair.

Two inventory rows below are corrected by r0031's reading of §7. Lemmas 28 and 30 are **not** about `P N` and not about `Fc(U)`: both are stated for *p-representability* over `Fp(U)`, Lemma 28 over §7.3's ideal completion of dyadic half-open intervals and Lemma 30 over §7.4's bifinite universal domain. Lemma 23 is therefore **not** Lemma 28's function-space conjunct. Also, the paper proves `F(X) = X + X` has **no** representation over `P N`, so a + conjunct there is false, not missing.

Theorem 12 is recoverable after all. The row below says its "axioms T" are never legible; `pdftotext -layout` extracts them cleanly — a continuous algebra `⊕ : E × E → E` with associativity, commutativity and idempotence, where `T♯` adds `s ⊕ t ⊑ s` and `T♭` adds `s ⊑ s ⊕ t`. It belongs to §5.3 with Lemma 13, not to the §7 group.

Round r0028 ran five agents in parallel and roughly doubled the development: 27 → 33 modules, 4440 → 8212 lines, 199 → 384 theorems, 9 → 15 numbered results. Every new result was kernel-audited with `#print axioms`: all depend only on `propext`, `Classical.choice`, `Quot.sound`, and none on `sorryAx`.

#	r0028 stream	Landed
1	`CoalescedSum.lean`, `Skeleton/Sum.lean`	`D + E` as a cpo (`sumSup`, `sumCpo`), then `lem10_sum`, `lem17_sum`, `lem17_smash` — **Lemma 10 at 6 of 6 conjuncts, Lemma 17 at 5 of 5**
2	`IdealCompletion.lean`	**Theorem 11**, both halves, on Mathlib's `Order.Ideal`; §5 unblocked

#	r0028 stream	Landed
3	`FinitaryProjectionPoset.lean`, `Skeleton/Section6b.lean`	`Fp(D)` and `Fc(D)` as posets, **Theorem 16** (algebraic-lattice conjunct), **Lemma 20**, and `IsClosure.domain_range` — Lemma 19 at the paper's strength
4	`MinimalUpperBounds.lean`	minimal upper bounds, `U`, `U^∞`, and `isPlotkinOrder_iff_mubClosure` — a characterization the paper does not state. **Theorem 18 still open**
5	`UniversalDomain.lean`	**Theorem 22** and **Lemma 23**, with *representable* defined from §7; opens the route to `D∞`

A name clash `lake build` could not catch. Streams 3 and 5 each defined `IsClosure.apply_sSup_of_directed` and `isClosure_sSup`. The build passed at 971 jobs because no module imported both; the clash appeared the moment anything did — here, an axiom audit importing the pair. `isClosure_sSup` was the *same* statement proved twice; the single copy now lives in `Skeleton/Section6.lean` beside `IsClosure`, which both modules already import, so no call site changed. The two `apply_sSup_of_directed`s were *different* statements sharing a name — one indexed by a set of the subtype `↥(im r)`, one by an ambient `D : Set α` with `D ⊆ im r` — so the subtype form was renamed `IsClosure.apply_sSup_val_image_of_directed`. **A green build is not evidence that parallel work composes**; importing every new module together is.

Row 5 counts lines matching `^(@[…] )?(theorem|lemma)` across the 37 modules.

The `sorry` burn-down (from r0026). The `sorry`s are deliberate scaffolding: fixed *statements* of outstanding results, confined to `ScottDomains/Skeleton/`, one file per agent worktree, so that three agents can prove them in parallel without any agent editing a declaration another depends on. Every other module remains `sorry`-free, and the count above is the burn-down metric — it goes 10 → 0. Round r0027, run as three agents in parallel, took it **10 → 1**.

#	Open statement	Result	State after r0027
1	`prop15`	Prop 15 — every bounded complete domain is bifinite	**proved**
2	`thm18`	Thm 18 — `D`, `D → D` domains ⟹ `D` bifinite	**`sorry`** — the paper gives no proof, citing Smyth [Smy83a]; the obstacle is recorded in the docstring
3	`lem19`	Lem 19 — the image of a closure is a domain	**proved**, via `IsClosure.rangeCompletePartialOrder`
4–7	`lem10_prod`, `lem10_smash`, `lem10_lift`, `lem10_strict`	Lem 10 — bounded completeness closed under `×`, `⊗`, `()⊥`, `→⊥`. The `→` conjunct is **already proved** (Thm 7's bounded-complete half, r0007)	**all four proved**; Lem 10 is then **5 of 6** conjuncts — `D + E` is not stated

#	Open statement	Result	State after r0027
8–10	`lem17_prod`, `lem17_lift`, `lem17_fun`	Lem 17 — bifiniteness closed under `×`, `()⊥`, `→`	**all three proved**; Lem 17 is then **3 of 5** conjuncts — `D ⊗ E` and `D + E` are not stated

The + conjuncts, closed in r0028. `CoalescedSum.lean` was 181 lines of ingredients with no `sSup` and no cpo instance, so there was no `D + E` to state a conjunct over. It now has both, and the guard is the membership condition `IsNonBotSum (sumCandidate (sumBase s))` — the defining predicate of the subtype — not directedness, so the defect fixed in `ScottHom` and then in `Smash` did not recur a third time. One wrinkle the smash did not have: `α ⊕ β` carries no `SupSet`, so a summand must be selected first; that selection is not a second guard, because a set with an upper bound at all lies in one summand.

The `smashSup` defect (r0027). `lem10_smash` was not merely open: as `smashSup` stood, it was **false**, and the kernel confirmed a refutation. `smashSup` branched its `dite` on the base being nonempty *and directed*, so a merely **bounded** non-directed base fell to the adjoined `⊥`, which is not even an upper bound. Witness: `D = Prop × Prop`, `E = Prop`, `s = {↑((True, False), True), ↑((False, True), True)}`, bounded by `↑((True, True), True)`. This is the same defect `ScottHom.lean` records having already hit for the function space. The repair branches on the coordinatewise supremum landing in `NonBotPair` — the condition under which it is an element of `D ⊗ E` at all, rather than a merely sufficient condition for it. `smashSup_of_directed` and `smashSup_of_empty` kept their statements and were reproved, which is the agreement claim stated in Lean rather than in prose, and `smashCpo` needed no change.

Kernel check on the r0027 merges (`#print axioms`): all ten proved statements plus `smashCpo`, `smashSup_of_directed` and `smashSup_of_empty` depend only on `propext`, `Classical.choice`, `Quot.sound` — `lem10_prod` and `lem19` do not even need `Classical.choice`. None depends on `sorryAx`. `thm18` does, as its `sorry` requires.

Row 2 counts only the paper's 30 **numbered** results (Theorems / Lemmas / Proposition 1–30). Row 3 counts the claims the paper makes **in prose** rather than as numbered results; these are paper content too, and all twelve are formally verified:

#	Paper claim	Where	Lean
1	"the compact elements [of `P N`] are just the finite subsets of `N`"	p. 9	`isCompactElement_iff_finite`
2	"`P N` … is a domain"	p. 9	`instance : Domain (Set X)`
3	"`D → E` is a … cpo"	Thm 7 proof	`instance : CompletePartialOrder (ScottHom α β)`
4	"… bounded complete … whenever `E` is"	Thm 7 proof	`instance : BoundedComplete (ScottHom α β)`
5	"step(s) … is continuous"	Thm 7 proof	`scottContinuous_stepFun`
6	"… and compact in the ordering on `D → E`"	Thm 7 proof	`isCompactElement_step`
7	"they form a basis for `D → E`"	Thm 7 proof	`instance : IsAlgebraic (ScottHom α β)`
8	every compact function is a *finite* join of step functions	Thm 7 proof, implicit	`exists_finite_isLUB_of_isCo`
9	"an embedding is an injection"	§3.1	`IsEmbeddingProjectionPair.i`
10	"a projection is a surjection"	§3.1	`IsEmbeddingProjectionPair.s`
11	"it is easy to check that `p_N` … is a finitary projection"	§3.1	`isFinitaryProjection_normal`
12	"the set of strict continuous functions `D → E` is also a cpo"	§2.1	`ScottDomains.strictHomCpo`

Six of those eleven are the body of **Theorem 7**, which is now **complete** — all four conjuncts of its conclusion (cpo, bounded complete, algebraic, countably based) are formally verified, as `ScottHom.isBoundedCompleteDomain_scottHom`.

It is proved under **weaker hypotheses than the paper states**: bounded completeness of `D` is never used. `D` need only be a domain. The function space is a cpo for any preordered `D`, algebraic when `D` and `E` are, bounded complete because `E` is, and countably based because `D` and `E` are.

The remaining theorems are supporting API: the ≪ calculus, the `compactsBelow` machinery, the pointwise order and suprema on `D → E`, and the step-function adjunction. The paper assumes or elides all of it.

Reference audit (r0020). Of the theorems in the development, 16 are never cited elsewhere. Nine of those are *terminal by design* — they are the paper's own claims, so nothing should cite them (`injective_embedding`, `surjective_projection`, the `isNormalIn_sUnion*` family and `mono_right` for Lemma 4, `singleton_bot_isNormalIn` for `{⊥} ∈ P(C)`, `isLeast_kleeneFix_le`, `eq_kleeneOperator_op`). The other six were speculative API written for callers that never appeared; they are **commented out in place**, each with a note on why it exists and what is instructive about it, and the build is unchanged — which also confirms that the three `@[simp]` ones among them were never firing implicitly.

The development is **45 modules, 14048 lines, 8 `sorry`, 0 other warnings** (measured by `scripts/counts.sh`). Seven of the eight `sorry`s are the newly *statable* Lemma 9 and Theorem 14 in `Skeleton/Recovered.lean`; the eighth is `thm18`. Counts of definitions, results and theorems are in the Progress table above — they are not repeated here, so that this section cannot drift out of step with it. What each round delivered:

#	Round	Module	Contents
1	r0003	`ScottDomains/WayBelow.lean`	way-below ≪ at `[Preorder α]`, 7 theorems; `x ≪ x ↔ IsCompactElement x` holds by `Iff.rfl`
2	r0004	`ScottDomains/Domain.lean`	`IsAlgebraic`, `Domain` (with the paper's countable-basis condition), `BoundedComplete`, 8 theorems, `DomainProp`
3	r0005	`ScottDomains/Powerset.lean`	the paper's `P N` (p. 9): compacts of `Set X` are exactly the finite subsets, hence `Domain (Set ℕ)` — the nondegenerate witness
4	r0006–r0007	`ScottDomains/ScottHom.lean`	the continuous function space `D → E`: `ScottHom`, the pointwise order, `CompletePartialOrder`, and `BoundedComplete` when `E` is — **Theorem 7's first sentence in full**
5	r0008	`ScottDomains/StepFunction.lean`	the single step function `step k e`: continuity (from `k` compact), the adjunction `step k e ≤ f ↔ e ≤ f k`, and compactness in `D → E` (from `e` compact)
6	r0009	`ScottDomains/FunctionSpaceDomain.lean`	**`D → E` is algebraic** — the paper's "they form a basis for `D → E`"; the two halves of `IsAlgebraic` use disjoint hypotheses (directedness needs only `E` bounded complete; the lub needs only `D`, `E` algebraic)
7	r0010	`ScottDomains/CompactFunctions.lean`	every compact function is a **finite** join of step functions — the finiteness half of the basis claim
8	r0011	`ScottDomains/FunctionSpaceCountable.lean`	`K(D → E)` is countable, hence `Domain (ScottHom α β)` — **Theorem 7 complete**
9	r0012	`ScottDomains/NormalSubposet.lean`	the normal-subposet relation ◁ and **Lemma 4**, all four parts
10	r0012–r0013	`ScottDomains/Projection.lean`	embedding–projection pairs, projections, the paper's "an embedding is an injection, a projection is a surjection"; **`im(p)` is a cpo** and **finitary projections**
11	r0014	`ScottDomains/FinitaryProjection.lean`	**Lemma 5** — the compacts of `im(p)` are `im(p) ∩ K(D)` (needs only that `p` is a projection), and `im(p) ∩ K(D) ◁ K(D)`

#	Round	Module	Contents
12	r0015	`ScottDomains/NormalProjection.lean`	`p_N(x) = ⨆{y ∈ N | y ⊑ x}`: continuous, a projection, and `im(p_N) ∩ K(D) = N` — half of **Theorem 6**'s correspondence, plus order preservation both ways
13	r0016	`ScottDomains/Theorem6.lean`	**Theorem 6** — `p_N` is finitary, `p_{im(p) ∩ K(D)} = p`, and the correspondence assembled
14	r0017	`ScottDomains/FixedPoint.lean`	**Theorem 1** — $\bigsqcup_n f^n(\bot)$ is the least fixed point of a continuous `f` on a cpo. Not Mathlib reuse; see the §2 table
15	r0018	`ScottDomains/UniformFixedPoint.lean`	**Theorem 3** — `fix` is the unique uniform fixed-point operator; `↓a` as a cpo
16	r0019	`ScottDomains/Product.lean`	`D × E` as a cpo (the one construction needing no case split) and **Lemma 8 parts 1–3**
17	r0021	`ScottDomains/Currying.lean`	**Lemma 8.4** — currying, `D → (E → F) ≅ (D × E) → F`, and with it **Lemma 8 complete**
18	r0022	`ScottDomains/EffectivePresentation.lean`	§3.2's **effective presentation** — the enumeration of the basis with its two decidability conditions
19	r0023	`ScottDomains/Lift.lean`	the **lift** `D⊥` as a cpo, on Mathlib's `WithBot`
20	r0024	`ScottDomains/StrictHom.lean`	the **strict function space** `D →⊥ E` as a cpo — needs no case split, since both branches of `ScottHom`'s `sSup` are strict
21	r0025	`ScottDomains/Smash.lean`	§4.3's **smash product** `D ⊗ E` — the non-bottom pairs with a new bottom adjoined, as a cpo
22	r0025	`ScottDomains/Bifinite.lean`	§6.1's **Plotkin order** and **bifinite** domain

§2 and §3 are now complete — the only §3 omission is the paper's *computable function*, which needs an r.e.-predicate notion Mathlib does not supply.

Dependency structure of the remaining definitions

#	Definition	Prerequisites	Status
1	smash product `D ⊗ E`	`Domain`, `OrderBot`	**✓ done, r0025**
2	bifinite / Plotkin order	`IsNormalIn` (◁, r0012)	**✓ done, r0025**
3	sum `D + E`	`Domain`	independent, ready
4	`D∞`	embedding–projection pairs (r0012)	independent, but large (inverse limits)
5	the three powerdomains	the **ideal completion** (Theorem 11) — *not* built	**blocked**; the three are independent of each other once it exists

Rows 1 and 2 were written as two modules and checked in a single build, which is the parallelism the structure actually admits — the bottleneck is the build-and-fix loop, not the writing.

Next: the sum `D + E`, then Theorem 11 to unblock the powerdomains, then Lemma 9 (the strict analogues of Lemma 8) and Lemma 10 (closure of bounded completeness).

Work counts

- **Reuse from Mathlib — no work (11):** poset, directed, cpo, ⊥, monotone, continuous, fixed-point operator, Schröder–Bernstein (2), algebraic lattice, product, λ-notation. (Theorem 1 was listed here in an earlier draft and has been removed: Mathlib's `OrderHom.lfp` is Knaster–Tarski over a complete lattice, not Kleene's $\bigsqcup_n f^n(\bot)$ over a cpo. It is proved in `ScottDomains/FixedPoint.lean`, so **29** numbered results needed proof, not 28.)
- **Generalize / adapt (4):** compact element (`IsCompactElement` — its *definition* is already stated at `[PartialOrder α]` and so applies to a dcpo verbatim; what is `CompleteLattice`-only is every lemma about it, so the dcpo API must be re-proved), sum, lift (`WithBot`/`Sum`), ideal completion (`Order.Ideal`).
- **Definitions to define — new (≈13; 4 done in r0003–r0004, 9 remaining):** way-below ≪ (**done** — `ScottDomains/WayBelow.lean`), algebraic cpo, **domain**, bounded-complete (**all three done** — `ScottDomains/Domain.lean`), embedding–projection pair, (finitary) projection, normal subposet, effective presentation, smash product, the three powerdomains (Hoare / Smyth / Plotkin), bifinite / Plotkin order, and `D∞`.
- **Theorems to prove (28):** 28 of the paper's 30 numbered results — Theorems 3, 6, 7, 11, 12, 14, 16, 18, 21, 22, 25, 26, 27, 29; Lemmas 4, 5, 8–10, 13, 17, 19, 20, 23, 24, 28, 30; Proposition 15. Only Theorems 1 & 2 come free from Mathlib.

Bottom line: ≈13 definitions to define + 28 results to prove, on top of 12 reused Mathlib foundations. After r0004: **9 definitions + 28 results** remain.

Lean column legend: ✓ reuse Mathlib (name given) · ~ partial (Mathlib has a related or lattice-only version — generalize) · ✗ define / prove (absent from Mathlib v4.32.2, confirmed by grep).

> Statements are paraphrased/de-garbled from the PDF (1990 Type-3 fonts render → as `!`, drop `fi` ligatures, mangle math), so read them as a guide to *what* each result is. Notation: ⊑ order, ⨆ directed sup, ⊥ bottom, ≪ way-below, `K(D)` compacts, `Fp(D)` finitary projections.

§2 Recursive definitions of functions

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
2.1	Def	—	**poset** (partially ordered set)	✓ `PartialOrder`
2.1	Def	—	**directed** subset `M`: every finite `u ⊆ M` has an upper bound in `M`	✓ `DirectedOn` / `IsDirected`
2.1	Def	—	**cpo**: poset in which every directed `M` has a lub ⨆`M`	✓ `CompletePartialOrder` (chains: `OmegaCompletePartialOrder`)
2.1	Def	—	**bottom** ⊥ (least element)	✓ `OrderBot`
2.1	Def	—	**monotone** function	✓ `Monotone` / `OrderHom`
2.1	Def	—	**continuous**: monotone, `f(⨆M) = ⨆f(M)` for directed `M`	✓ `ScottContinuous`
2.1	Thm	1	**Fixed-Point Theorem**: `f : D → D` continuous ⟹ least fixed point ⨆_n `fⁿ(⊥)`	✓ **proved** (r0017) — `ScottDomains.theorem1`. **Not** Mathlib reuse: `OrderHom.lfp` is Knaster–Tarski over a *complete lattice* with only monotonicity, a different theorem
2.2	Thm	2	**Schröder–Bernstein** for sets	✓ `Function.schroeder_bernstein` (`SetTheory/Cardinal/SchroederBernstein.lean` — the name in an earlier draft of this row, `Function.Embedding.schroederBernstein`, does not exist
2.3	Def	—	**fixed-point operator** (uniform)	✓ `OrderHom.lfp` / `LawfulFix`
2.3	Thm	3	The standard operator is the **unique uniform** fixed-point operator	✓ **proved** (r0018) — `ScottDomains.theorem3`; a fixed point operator is formalized as a family over every `CompletePartialOrder` in a universe

§3 Effectively presented domains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
3.1	Def	—	**compact element** `x`: `x ⊑ ⨆M` (dir.) $\Longrightarrow$ `x ⊑ y` some `y∈M`; `K(D)`	~ `IsCompactElement` — def is `[PartialOrder]`, so reusable on a dcpo; its lemmas are all `CompleteLattice`-only
3.1	Def	—	**way-below** ≪ (approximation relation behind compactness)	✓ `ScottDomains.WayBelow` (r0003) — `x ≪ x ↔ IsCompactElement x` by `Iff.rfl`
3.1	Def	—	**algebraic** cpo: `x = ⨆{x'∈K(D) : x'⊑x}` (directed)	✓ `ScottDomains.IsAlgebraic` (r0004)
3.1	Def	—	**domain** = algebraic cpo **whose basis `K(D)` is countable** (the paper's definition, p. 9 — the countability condition was missing from an earlier draft of this row)	✓ `ScottDomains.Domain` (r0004)
3.1	Def	—	**bounded complete**: `⊥` + every bounded subset has a sup — a *separate* predicate; the paper composes them as "bounded complete domain" (Thm 7, Lem 10, Lem 13, Thm 14), which is the literature's *Scott domain*	✓ `ScottDomains.BoundedComplete` (r0004); the compound is `[Domain α] [BoundedComplete α]`
3.1	Def	—	**(countably based) algebraic lattice**	✓ `IsCompactlyGenerated` (`+CompleteLattice`)
3.1	Def	—	**embedding–projection pair** `(g, f)`	✓ `ScottHom.IsEmbeddingProjectionPair` (r0012)
3.1	Def	—	**projection**; **finitary projection** `p`: `p∘p=p⊑id`, `im(p)` a domain	✓ `ScottHom.IsProjection` (r0012), `ScottHom.IsFinitaryProjection` (r0013) — `im(p)` carries a `CompletePartialOrder` via `IsProjection.rangeCompletePartialOrder`
3.1	Def	—	**normal subposet** / substructure	✓ `ScottDomains.IsNormalIn`, notation ◃ (r0012)
3.1	Lem	4	`⟨P(C), ◃⟩` of substructures is a cpo with `{⊥}` least	✓ **proved** (r0012), all four parts
3.1	Lem	5	`p` finitary projection $\Longrightarrow$ compacts of `im(p)` are `im(p) ∩ K(D)`, and `im(p) ∩ K(D) ◃ K(D)`	✓ **proved** (r0014) — `IsProjection.isCompactElement_iff` (needs only *projection*) and `IsFinitaryProjection.isNormalIn_compacts`
3.1	Thm	6	Isomorphism: normal substructures ≅ `Fp(D)` (finitary projections)	✓ **proved** (r0015–r0016) — `ScottDomains.theorem6`
3.1	Thm	7	`D,E` bounded-complete domains $\Longrightarrow$ `D → E` bounded-complete domain	✓ **proved** (r0006–r0011) — `ScottHom.isBoundedCompleteDomain_scottHom`; `D` bounded complete is not needed

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
3.2	Def	—	**effective presentation** `d : ℕ → K(D)`; **effectively presented domain**	✓ `ScottDomains.EffectivePresentation` (r0022). The paper's **computable function** is ✓ **defined** (r0031) in `ComputableFunction.lean`, on Mathlib's **`REPred`** (`Mathlib/Computability/RE.lean:157`). An earlier draft of this row said "Mathlib v4.32.2 has no `RePred` or equivalent (grep finds none)" — that grep used the wrong capitalization, and the claim was false. Two caveats recorded there: `decidableLE` is too weak to prove anything computable (a `Decidable` instance may be `Classical.dec`), so results need an explicit `RecursiveLE` hypothesis; and composition awaits `REPred` closure under `∧` and `∃`, which Mathlib's five-lemma API does not supply

§4 Operators and functions

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
4.1	Def	—	**product** `D × E`	✓ `Prod` order from Mathlib; the **cpo instance** is `ScottDomains.instCompletePartialOrderProd` (r0019) — Mathlib has `Prod.supSet` and `isLUB_prod` but no cpo instance
4.2	Def	—	**Church's λ-notation** (continuous abstraction)	✓ `OrderHom` / ωCPO `ContinuousHom`
4.3	Def	—	**smash product** `D ⊗ E`	✓ `ScottDomains.Smash` + `smashCpo` (r0025)
4.4	Def	—	**sum** `D + E`; **lift** `D⊥`	both ✓ — lift `ScottDomains.liftCpo` on `WithBot` (r0023); the coalesced sum `CoalescedSum` with `sumSup` and `sumCpo` (r0028), its `sSup` guarded on landing in `NonBotSum`
4.x	Lem	8	`D×E ≅ E×D`; `(D×E)×F ≅ D×(E×F)`; `D→(E×F) ≅ (D→E)×(D→F)`; `D→(E→F) ≅ (D×E)→F`	✓ **proved** — `prodComm`, `prodAssoc`, `scottHomProd` (r0019); `scottHomCurry` (r0021)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
4.x	Lem	9	Iso laws over `D,E,F`: `D⊗E ≅ E⊗D`; `(D⊗E)⊗F ≅ D⊗(E⊗F)`; `(E⊕F) ∘→ D ≅ (E ∘→ D) × (F ∘→ D)`; `D ∘→ (E ∘→ F) ≅ (D⊗E) ∘→ F`; `D⊗(E⊕F) ≅ (D⊗E)⊕(D⊗F)`; `D⊥ ∘→ E ≅ D → E`	✓ **proved** (r0034) — six named ≃o under `ScottDomains.Isomorphism`: `smashComm`, `smashAssoc`, `coalescedSumCopair`, `smashCurry`, `smashDistribCoalescedSum`, `liftStrictHomIso`. Items 3 and 5 are **false as printed** and are kernel-checked negations, `lem9_3_printed_false` and `lem9_5_printed_false` in `Isomorphism/Counterexample.lean`. The witnesses are **not** the cardinality argument of `docs/StatementRecovery.md` (`D = E = Prop`, `F = Prop × Prop`), which needs `Fintype` instances for `WithBot` of a subtype plus an enumeration of the strict monotone maps: they are `PUnit`-based (`D = PUnit, E = F = Prop` for item 3; `D = Prop, E = PUnit, F = Prop` for item 5) and separate the sides by the coarsest invariant an order isomorphism must preserve — one element versus more than one — discharged by `Equiv.subsingleton`. Each witness is chosen so the *corrected* law survives it, so the separation is specific to the misprint. `∘→` is the strict function space; it and `→` both extract as `!`
4.5	Lem	10	`D,E` bounded complete ⟹ `→, ∘→, ×, ⊗, ⊕, +, ()⊥` bounded complete	✓ **7 of 7** (r0034) — `→` r0007; `×`, `⊗`, `()⊥`, `∘→` r0027 (`Skeleton/Lemma10.lean`); `⊕` r0028 (`Skeleton/Sum.lean`); `+` as `ClosureProperties.lem10_separated`, via `D + E = D⊥ ⊕ E⊥` — §4.4's own definition, `+` being a different operator from `⊕`. Now also stated as **one** theorem, `ClosureProperties.lemma10`, a conjunction over the paper's operator list, so the conjunct count is kernel-checked rather than prose. The docstring paraphrase in `Skeleton/Lemma10.lean` is six wide because `⊕` prints blank under `pdftotext` — a fourth instance of the glyph-dropping mechanism
4.5	Thm	11	**Ideal completion** of a countable pre-order is a domain (all domains so arise)	✓ **proved** (r0028) — `IdealCompletion.thm11` and `thm11_converse`, on Mathlib's `Order.Ideal`

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
4.5	Thm	12	Initiality of a continuous algebra satisfying the axioms **`T♮`** (natural — not an undecorated `T`; `pdftotext` renders `♮/♯/♭` as `\/]/[`)	✓ **proved** (r0032) — `ContinuousAlgebra.thm12_plotkin`, `thm12_smyth`, `thm12_hoare`: `T♮ → D♮`, `T♯ → D♯`, `T♭ → D♭`, each `∃! h`, existence *and* uniqueness, factoring through `{|·|}` rather than the weaker principal ideals. `[IsAlgebraic D]` is the whole hypothesis — countability of `K(D)` is never used, and bounded completeness is never needed. Each free algebra is also proved a model of its own theory, a gap the plan had not named
4.5	Lem	13	`D` bounded complete ⟹ powerdomains `D]`,`D[` bounded complete	✓ **proved** (r0031) — `PowerdomainBC.lem13_hoare`, `lem13_smyth`, in the paper's wording. `D♭` needs no bounded-completeness hypothesis at all; there is no convex conjunct to prove
4.5	Thm	14	**Plotkin's SFP characterization**: `D` bifinite ⟺ `D` a domain and `K(D)` a Plotkin order (*not* "equivalent characterizations of an (algebraic/BC) domain" — that reading of the row was wrong; the theorem is two items about bifiniteness)	✓ **proved** (r0036) — `Recovered.thm14`, both directions, from `SFP.thm14_forward` and `SFP.thm14_converse` in `ScottDomains/SFP.lean`. Of the four gaps r0034 recorded in `thm14`'s docstring, two were real and two were false constraints. **Real:** gap 2, the bridge `Set.range ⇑(toFp hN) = N` for finite normal `N`, proved as `SFP.range_normalHom_of_finite` — for finite `N` the directed set `N ∩ ↓x` contains its own greatest element, which is `p_N(x)`; for infinite `N` the statement is false. Gap 4, the two finite-combinatorial lemmas, proved as `SFP.exists_upperBound_of_finite_subset` and `SFP.exists_greatest_of_finite`. **False constraints:** gap 1 (the `FpLattice` section sits at `[Domain α]`) never binds, because on a finite image `im(p) ∩ K(D) = im(p)`, so the forward direction runs entirely in `D` and touches no `Fp(D)` machinery; gap 3 (`IsLUB` in `↥(Fp α)` weaker than in `ScottHom α α`) never arises, because leastness of `id` is proved in the function space directly from algebraicity. One ingredient the r0034 note omitted **is** needed: `M.Nonempty`, since `IsCompactElement` quantifies over nonempty directed sets and Mathlib's `DirectedOn` is vacuous on `∅` — supplied by `SFP.isFinitaryProjection_const_bot`

§5 Powerdomains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
5.1	Def	—	**powerdomain** (non-deterministic outcomes)	✓ each of the three is `IdealCompletion (Pf K(D))` under its pre-order (r0029)
5.2	Def	—	**Hoare (lower)**, **Smyth (upper)**, **Plotkin (convex)** powerdomains	✓ `ScottDomains.Hoare.Powerdomain`, `Smyth.Powerdomain`, `Plotkin.Powerdomain` (r0029), each with its `Domain` instance from Theorem 11 and its compacts characterized as the principal ideals
5.3	—	—	Universal & closure properties (see Lem 13, 28, 30)	✗ prove — r0030 wave A

§6 Bifinite (SFP) domains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
6.1	Def	—	**Plotkin order** / **bifinite (SFP) domain**	✓ `ScottDomains.IsPlotkinOrder`, `IsBifinite` (r0025)
6.1	Prop	15	Every bounded-complete domain is bifinite	✓ **proved** (r0027) — `ScottDomains.prop15`, the paper's own proof over `lubClosure u`
6.1	Thm	16	`D` bifinite ⟹ `Fp(D)` is an algebraic lattice	**settled in all three directions** as of r0034: the algebraic-lattice conjunct ✓ **proved** (r0028) as `ScottDomains.thm16`; the `Fp(D) ↪ (D → D)` embedding conjunct ✗ **refuted** (r0032) in `FinitaryProjectionEmbedding.lean`, kernel-checked, with the sketch's exact error identified; and the conjunct ✓ **holds under a named hypothesis** — `Section62.thm16_positive` with `HasGreatestStableNormal`, plus `thm16_positive_isEmbeddingProjectionPair` giving the pair itself (r0034). Bounded complete domains satisfy the hypothesis

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
6.2	Lem	17	`D,E` bifinite ⟹ `→, ∘→, ×, ⊗, ⊕, +, ()⊥, D♮, D♯, D♭` bifinite	✓ **10 of 10** (r0034), with one qualification — `×`, `()⊥`, `→` r0027; `⊗`, `⊕` r0028; and r0034 adds `lem17_separated` (+), `lem17_strictFun` (`∘→`) and `lem17_hoare/lem17_smyth/lem17_plotkin` for `D♭/D♯/D♮`. The three powerdomain conjuncts had been dropped from the extraction with their glyphs, not for want of the objects — all three powerdomains have existed since r0029. One lemma covers all three: the obvious Hoare-maximal candidate `{n ∈ N | ∃ y ∈ w, n ⊑ y}` is greatest for `⊑♭` but its Smyth conjunct fails for `⊑♮`; the image `p_N[w]` is greatest for all three (`isNormalIn_image_principal` over `SelectsGreatest`). Now stated as one theorem, `ClosureProperties.lemma17`. **Qualification:** the `→` and `∘→` conjuncts carry `[BoundedComplete β]`, inherited from the step-function decomposition — stronger than the paper states, and §6 exists precisely to avoid bounded completeness. Removing it is a real open item

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
6.2	Thm	18	If `D` and `D → D` are domains, then `D` is bifinite	✗ prove — **the development's only remaining `sorry`**, and **no longer blocked on an unobtainable source**. [Smy83a] is needed for attribution only: r0034 recovered a complete proof of the same statement from **Jung 1989, which is on disk**, and mapped it to five steps in `Section62.lean`. Step 5 is `isBifinite_iff_mubClosure` (r0028). r0036 proved **steps 2 and 3** in `JungSFP.lean`: `lemma213` and `thm214`, the bifurcation into bifinite-or-(vii), and `lemma217`, the cardinality argument, spending countability exactly once via `Function.cantor_surjective` against `Domain.countable_compacts`. What unblocked three failed rounds was a missing bridge, not a tactic: the development states minimal upper bounds relative to `K(D)`, while Jung applies minimal-ity to bounds not known to be compact — `isCompactElement_of_minimal_upperBounds` (his Prop 1.9 for `IsAlgebraic`) closes the gap, and without it `f_A`'s monotonicity is false. **`IsLDomain` is not needed**: Lemma 2.17 uses only condition (vii) of Jung's Theorem 2.10, so `thm214`'s second disjunct is (vii)-minus-existence, not the literature's "algebraic L-domain". Remaining: Jung's Lemma 1.29 (property M for pairs ⟹ for all finite sets, the cheapest and the one that makes `lemma217` directly consumable), step 4 (Lemma 2.2, needing Rado's Selection Theorem and Corollary 1.36), and step 1 (Theorem 1.37, a separate development over ordinal-indexed codirected nets). r0031's (★) is **equivalent** to Theorem 18, not below it, and Jung's proof never passes through it
6.2	Lem	19	closure `r:D→D` (`r∘r=r⊒id`) ⟹ `im(r)` is a domain	✓ **proved** — `lem19` (r0027) gives the cpo structure; `IsClosure.domain_range` (r0028) gives the paper's full strength, `im(r)` a domain with basis `{r(k) | k ∈ K(D)}`
6.2	Lem	20	`D` domain ⟹ `Fc(D)` (finitary closures) is a cpo	✓ **proved** (r0028) — `ScottDomains.lem20`, over `Fc α` with the pointwise order

§7 Recursive definitions of domains (universal domain, `D∞`)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
7	Def	—	**recursive domain equation**; **universal domain**. (Earlier drafts of this row said "D∞ (inverse limit)" — §7 builds no inverse limit; see the note under Progress)	✓ `Recursive.Solves` / `IsSolvable`, and `Recursive.IsUniversal` / `IsUniversalRetract` for the paper's two phrasings (r0029)
7	Thm	21	`F` representable over cpo `U` ⟹ a domain `D` with `D ≅ F(D)`	✓ **proved** (r0029) — `ScottDomains.Recursive.thm21`; with `IsRepresentable₂.diag` and Lemma 23 it yields `recursiveDomain_funSpace`, the reflexive domain `D ≅ (D → D)`
7	Thm	22	Any countably-based algebraic lattice `L`: a closure `r : P(ℕ) → L`	✓ **proved** (r0028) — `ScottDomains.thm22`, with `thm22_of_isCompactlyGenerated` the Mathlib-vocabulary form
7	Lem	23	The function-space operator is representable over `P(ℕ)`	✓ **proved** (r0028) — `ScottDomains.lem23`
7	Lem	24	`U` cpo; × and → representable over `U` ⟹ (setup for universality)	✓ **proved** (r0032) — `Universality.lem24`, concluding also that `D`, `E` are closures of `U`, which its own proof needs
7	Thm	25	`U` non-trivial domain representing ×,→ ⟹ `U` **universal**	✓ **proved** (r0032) — `Universality.thm25`, with `thm25_powerset` instantiating it at `P N` and `thm25_isUniversal` in `Recursive.IsUniversal` form. Proved at **cpo** strength: no step spends algebraicity or countability, so it is stronger than the paper's statement
7	Thm	26	Any signature `(s₁,…,s_n)`: **combinations** `F₁,…,F_n` over `S`, `K`, `fst`, `snd` into which every continuous algebra carried by a retract of `D` embeds as a subalgebra. An earlier draft of this row said "combinators solving the equations" — §7.2 solves nothing; it is an Engeler-style universal-algebra result and `Recursive.Solves` is the wrong vocabulary	✓ **proved** (r0034) — `Combinator.thm26`, with `thm26_subalgebra`, `thm26_retract`, `exists_lambdaModel_of_thm25`, and `Comb`/`combEval` so the `Fᵢ` are genuinely combinations. Signature indexed `Fin n → ℕ`, because `Fᵢ` reads its slot by applying `snd` exactly `i` times. **Carries `hs : ∀ i, 0 < s i`**: the theorem is *false* for a signature admitting arity 0, which the paper explicitly allows — two one-point retracts with different constants must embed onto the same `Fᵢ`, and `fst(ψ(x)) = x` forces them equal. That argument is prose in the docstring, not kernel-checked

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
7	Thm	27	Any bounded-complete `D`: a projection of the universal domain onto `D`	✓ **proved, unconditionally** (r0036) — `Atomless.thm27`, with no hypothesis beyond `[Domain D]` `[BoundedComplete D]`, and non-vacuity compiled in (`example := thm27 Dyadic.U`). **Neither Vaught's theorem nor a Boolean algebra is used**, correcting two claims this file previously made. `IsNormallyRepresented ↥(compacts D)` is discharged by `Atomless.isNormallyRepresented`: what the theorem consumes is three properties of a map `ψ : K(D) → U₀` — order embedding into the superset order, `ψ ⊥ = [0,1)`, and `ψ a ∩ ψ b` empty when `{a,b}` is unbounded and `ψ(a ⊔ b)` when bounded — and the third is also what makes `range ψ` normal, so the paper's unproved final clause is four lines. `ψ` reads points of `S` as binary digit sequences and adjoins `enum n` to a branch only when `Legal`, which demands boundedness *and* that the join drag in no earlier `enum i` the branch does not carry; that second conjunct is what keeps `ψ a` a finite union of intervals. Same mathematics as the paper's atom splitting, stated on one element rather than on the atoms of a finite subalgebra. The Mathlib measurement stands — v4.32.2 has **zero** occurrences of `IsAtomless` — so the paper's own route would have had to be built from nothing. The conditional form is retained as `Dyadic.thm27`, which builds `e : ScottHom D U`, `p : ScottHom U D` with `p ∘ e = id`, `e ∘ p ⊑ id`, going directly between `D` and `U` rather than through `normalHom/im(p)`. Normality is spent in exactly one place, making `{k | ψ k ∈ I}` directed; `BoundedComplete D` is consumed inside the hypothesis. **The open step is `IsNormallyRepresented`**, which needs Vaught's theorem — the countable atomless Boolean algebra is unique up to isomorphism. Mathlib v4.32.2 has **zero** occurrences of `IsAtomless` and none in `ModelTheory/`. Two smaller steps also remain: that `B = U₀ ∪ {∅}` is a Boolean algebra (`isBasic_inter` is the intersection half, proved), and that the embedding cuts down to a *normal* subposet, which the paper asserts without proof. `prop15` and `thm22` are **not** on this route — §7.3 cites no earlier result of the paper

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
7	Lem	28	Operators **`→, ↛, ×, ⊗, +, ⊕, ()⊥, ()♯, ()♭`** — **nine**, read off a 600 dpi rendering of printed page 42 in r0036, since `pdftotext` emits `!, !, , ; +, ; ()?, ()], ()[` for the whole list. Two earlier drafts of this row were wrong in opposite directions: seven operators, then nine including `()♮` and omitting `↛`. `()♮` is **not** here — §7.4 opens by saying it cannot be representable over `U`, as it does not preserve bounded completeness. All nine are **p-representable over `Fp(U)`**, *not* the closure notion of `IsRepresentable` over `Fc(U)`	~ **2 of 9, at the paper's own notion** (r0036) — `PRep.rep_prod` (`×`) and `PRep.rep_lift` (`()⊥`) at `IsPRepresentable` over `Fp U`, with `PRep.Lemma28` the nine-fold conjunction and `lemma28_of` taking nine named hypotheses, so the count is kernel-checked rather than prose. This is a *smaller* number than the 3 of 9 r0034 recorded and a better position: **those three do not transfer, and that is now kernel-checked.** `Fp` adds an obligation `Fc` never had (`IsFinitaryProjection p` carries `Domain ↥(Set.range p)`), and the hypothesis is incompatible — `Combinator.Retracts` gives `id ⊑ gr ∘ fn` where the projection scheme needs `gr ∘ fn ⊑ id`, and `gr_fn_eq_of_both` proves holding both forces `U ≅ V`. `⊗` and `⊕` are **no longer refuted**: a projection has `p ⊥ = ⊥`, so r0034's three-chain counterexample does not apply. `isFinitaryProjection_sSup` is the keystone for the remaining seven — over a domain the directed supremum of finitary projections is finitary, so least upper bounds in `Fp(D)` are pointwise, which every conjunct's continuity proof needs; each is then 120–180 lines. Two docstring claims corrected: `PRepresentable.lean` said Lemma 28 is the `Fc` notion (§7.3's "for the remainder of this section" refutes it), and `CombinatorRep.lean` claimed `()♯/()♭` are undefinable on `Cpo` — `smythOp/hoareOp` compile, the `[Domain D]` being spent only on countability of the result

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
7	Thm	29	`D` bifinite $\Longrightarrow$ `D⁺` bifinite; **and** solving `D ≅ D⁺` — record as **split**, the two sentences have different evidential status	~ **first sentence proved** (r0034) — `BifiniteUniversal.thm29`, with `Plus D = IdealCompletion (MPair ↥(compacts D))`. Two **kernel-checked defects in §7.4**: the printed relation is *not reflexive* (`b = (⊥,∅)` fails `b ⊢ b`, as does `(false,{true})` over `Bool`), so it is the strict part and the order is its reflexive closure — exactly the identification §7.4 then performs by hand; and the worked example reverses its own definition (`b ⊢ a` where both papers give `a ⊢ b`). A rival repair (Smyth order on covers) also contains the printed relation and agrees at stages 0–2; **the paper's own element counts select between them** — 1, 2, 5, **20** for the adopted reading against 1, 2, 5, **21** for Smyth. **`V` is built** (r0036) — `Colimit.V`, with `domain_V`, `isBifinite_V` and `isPlus : V ≃o Plus V`, so `D ≅ D⁺` is solved. `Recursive.thm21` indeed does not apply, `D ↦ D⁺` being a functor on domains rather than an operator representable over a fixed cpo; the colimit is taken at the level of countable posets and `IdealCompletion.thm11` is applied once at the end, so `V` needs no cpo construction. Shape: the `Antisymmetrization` of a pre-order on `Σ n, Stg n` — forced, since `MPair (MPair A)` does not typecheck (`MPair` is a pre-order) — with `Nat.leRecOn` giving transport with **zero casts**, so the predicted dependent-transport cost did not materialize. **A third printed defect in §7.4**, kernel-checked as `Colimit.stgEmb_ne_mk_eta`: the paper (and `BifiniteUniversal.lean`) say the colimit is along `eta`, `x ↦ (x, {x})`, and **that colimit is not a fixed point of `M`** — reading `(x, u) ∈ M(A_N)` one stage later gives base `[(x, {x})]` where the colimit map needs `[(x, u)]`, and they agree only when `↑u = ↑x`, failing at §7.4's own `b = (⊥, ∅)`. The correct chain applies `M` to the previous connecting map; the two first differ at stage 1→2, where both values lie among §7.4's five elements of `I⁺⁺`, so Figure 4 does not discriminate them and the 1, 2, 5, 20 count is unaffected. Still open: the universality extension step, carried as `Colimit.Thm29Second` with the gap located

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
7	Lem	30	**Ten** operators — Lemma 28's nine **plus** **`()♮`**, the convex powerdomain, which is the whole point of §7.4 — **p-representable** over the universal bifinite `V`. An earlier draft of this row said nine, copying Lemma 28's list	✗ prove — **0 of 10**, but **statable for the first time**: `V` was built in r0036 (`Colimit.V`, with `V ≃o V⁺`), and the blocker this row recorded is gone. Only the → conjunct is currently type-correct, carried as `Colimit.Lem30Arrow`; the other nine do not yet exist as `Cpo → Cpo` operators in the `Fp` setting. The notion itself exists: `PRepresentable.IsPRepresentable/IsPRepresen` over `↥(Fp U)` (r0034), with `eq_id_of_mem_Fp_of_mem_Fc` proving `Fp` and `Fc` meet only at `ScottHom.id` — so the distinction from Lemma 28's `IsRepresentable` is kernel-checked, not merely documented. Also landed: `isProjection_repOf`, the projection half of the paper's conjugation recipe. No longer blocked behind the unobtainable `[Gun87]` — `V` was built directly

---

Tally (matched): ✓ reuse Mathlib ≈ **12** (poset, directed, cpo, ⊥, monotone, continuous, fixed-point Thm 1 & operator, Schröder–Bernstein, algebraic lattice, product, λ-notation) · ~ partial ≈ **4** (compact element, sum/lift, ideal completion Thm 11, Thm 22) · ✗ define / prove ≈ **44**, of which 4 are done (≪ r0003; algebraic, domain, bounded-complete r0004) → **40 remaining —** the bifinite / powerdomain / `D∞` development and all 28 numbered results.

What is done, and what is next, is listed under **Progress** at the top of this file.